

Turn On LED Light

Turn On LED Light

1. Learning Objectives
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. Parameter Settings
 - 2.1 Set Port Parameters
 - 2.2 Set Pin Parameters
 - 2.3 Save and Update Configuration File
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

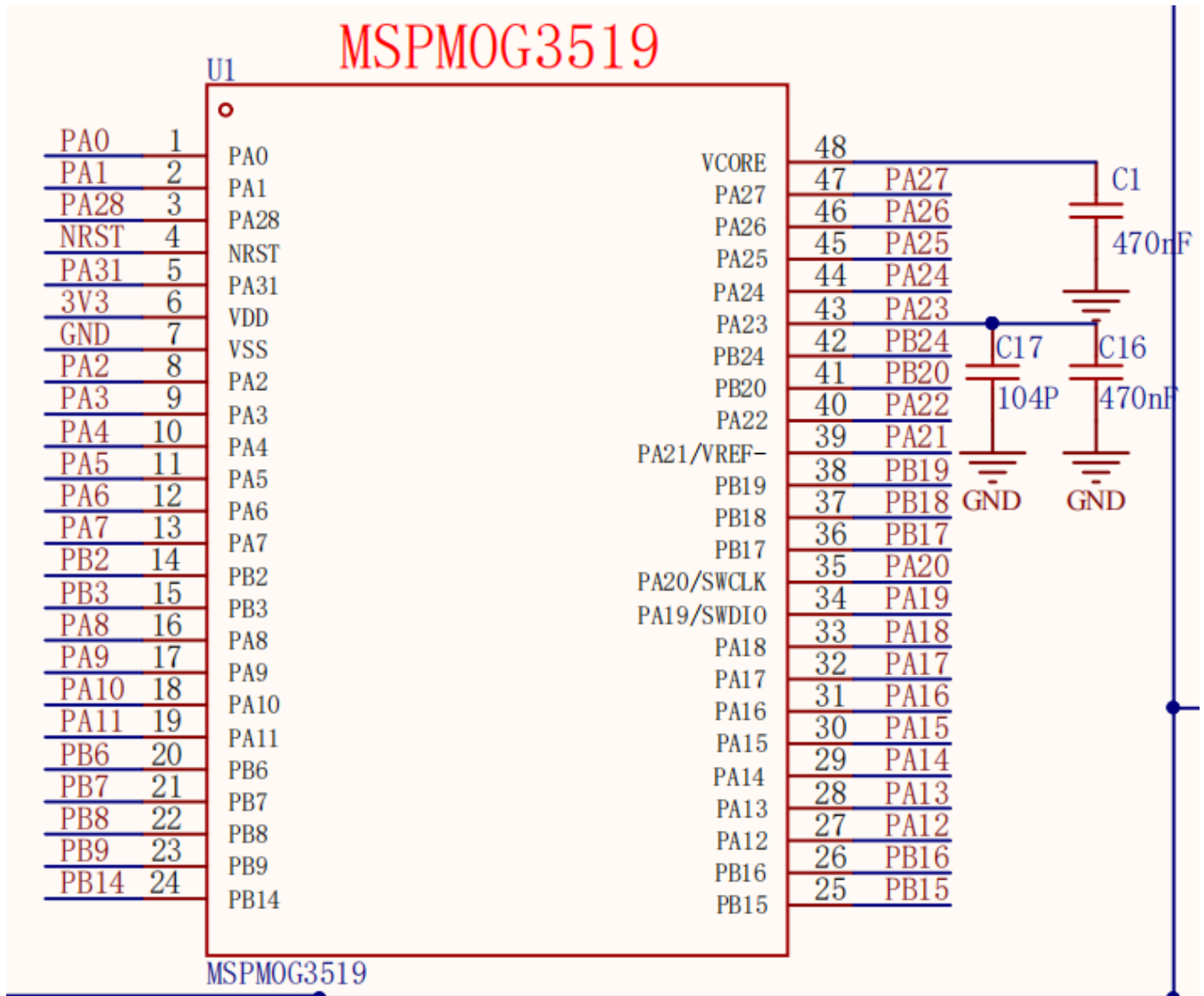
1. Learn the basic use of MSPM0G3519 motherboard pins.
2. Understand how to control the onboard LED light.

2. Hardware Setup

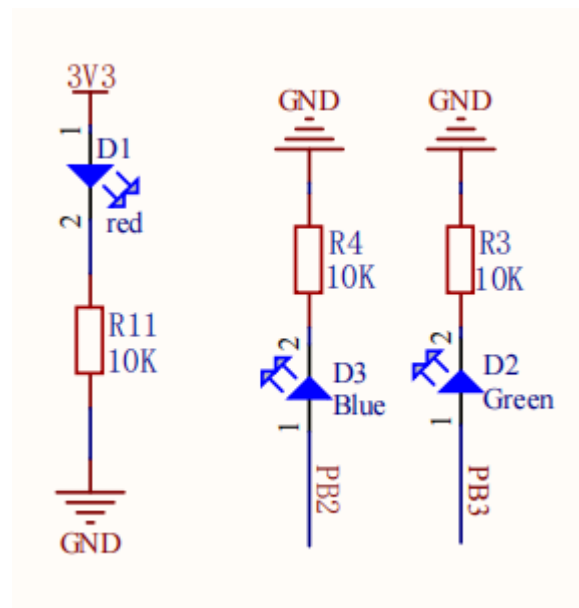
This course does not require additional hardware. You can directly use the onboard LED light on the MSPM0G3519 motherboard.

We have set up two LED user lights on the motherboard. Users can customize their functions. The following uses **LED1** as an example.

MSPM0G3519 Main Control Diagram



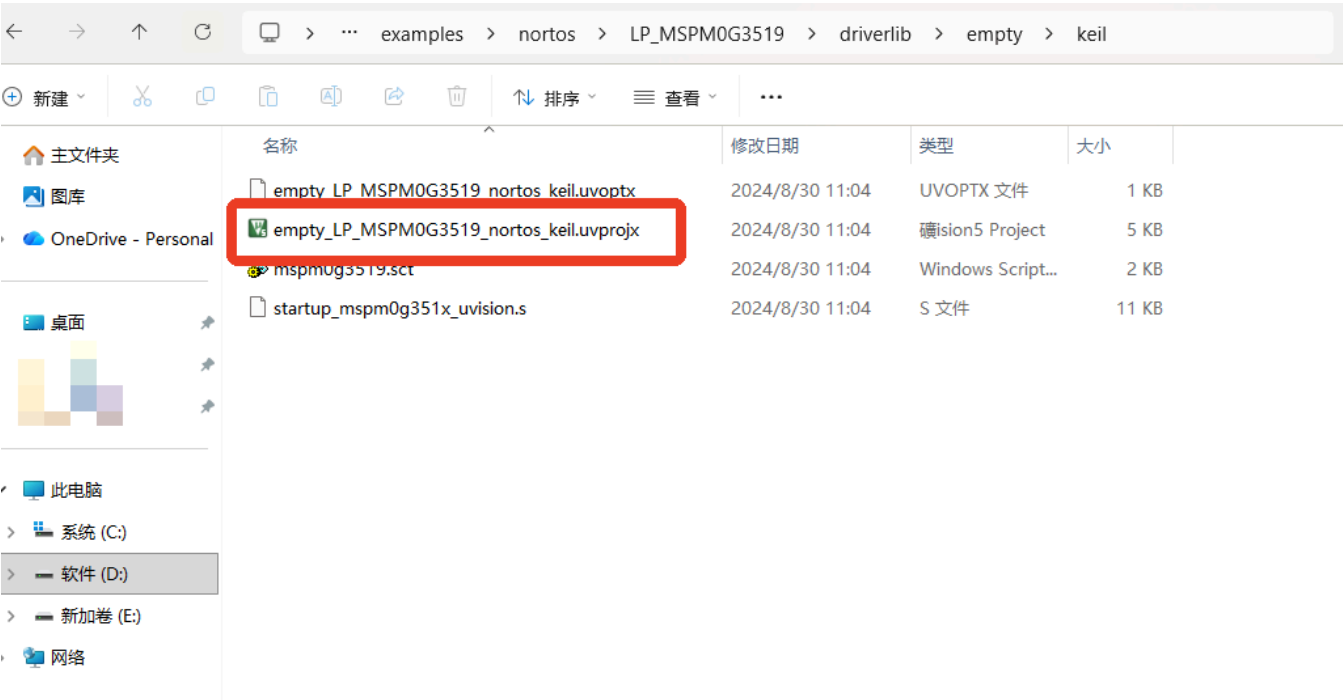
LED Part Schematic



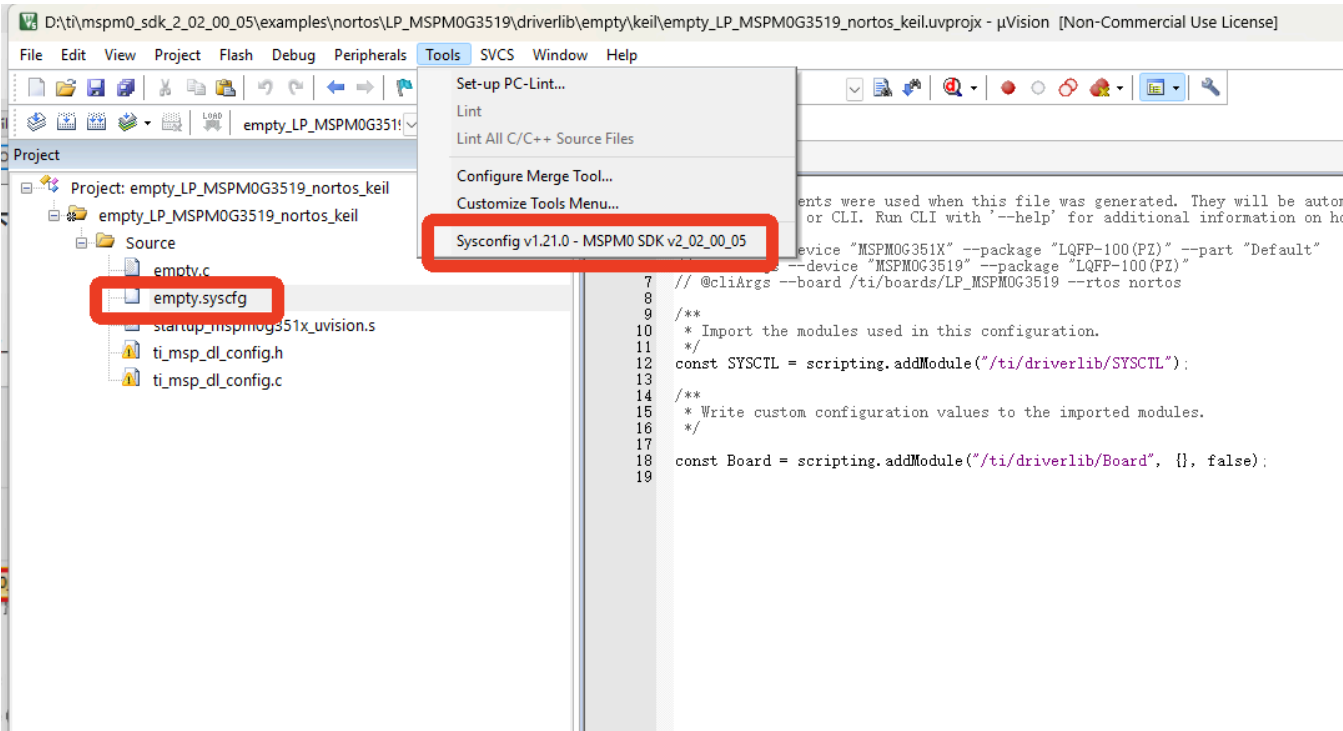
3. Experiment Steps

1. Open SYSCONFIG Configuration Tool

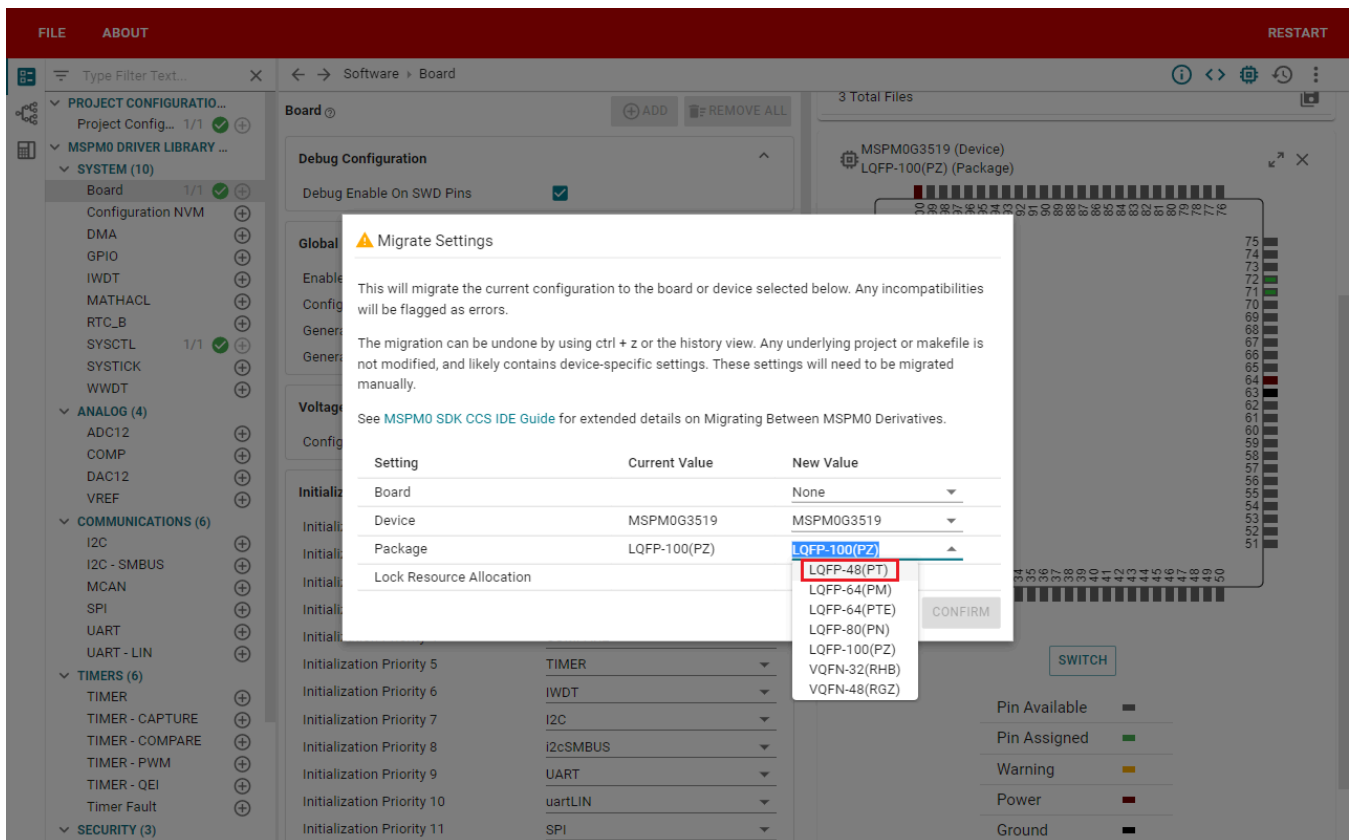
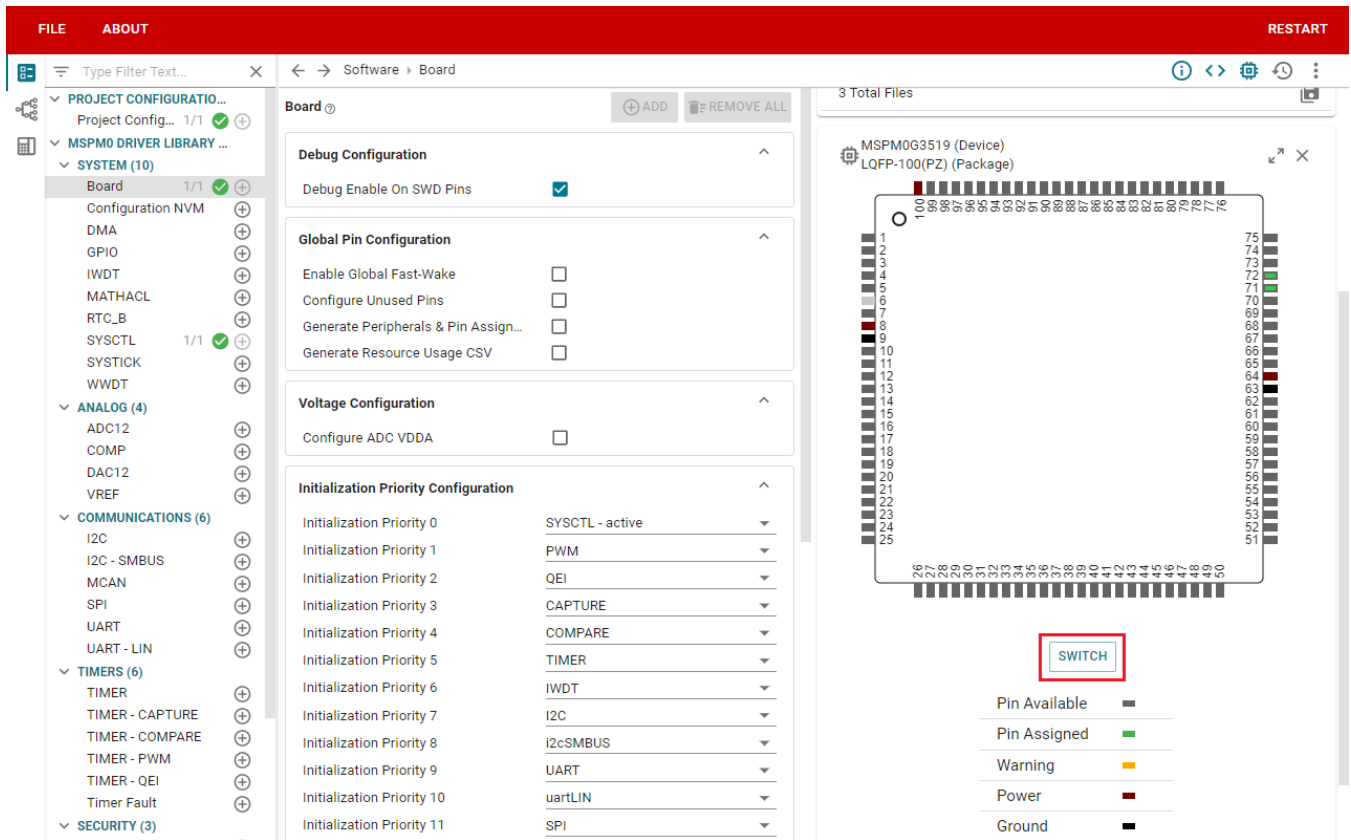
Open the empty project from the SDK in KEIL.



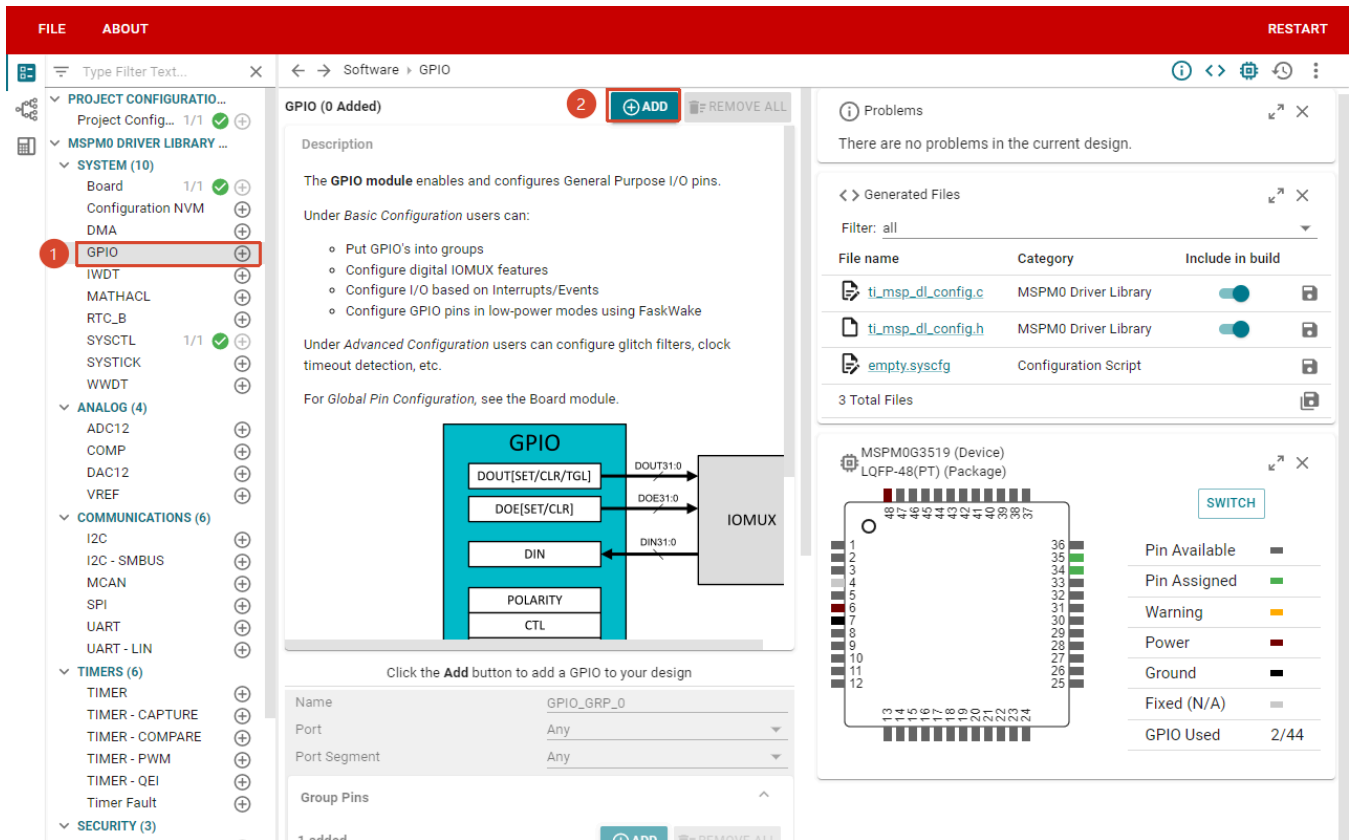
After selecting and opening it, open the empty.syscfg file in the KEIL interface. **With the empty.syscfg file open**, then open the SYSCONFIG GUI interface from the Tools menu.



In SYSCONFIG, Modify chip pin package



select MCU peripherals on the left, find the GPIO option and click to enter. Click ADD in GPIO to add a GPIO group.

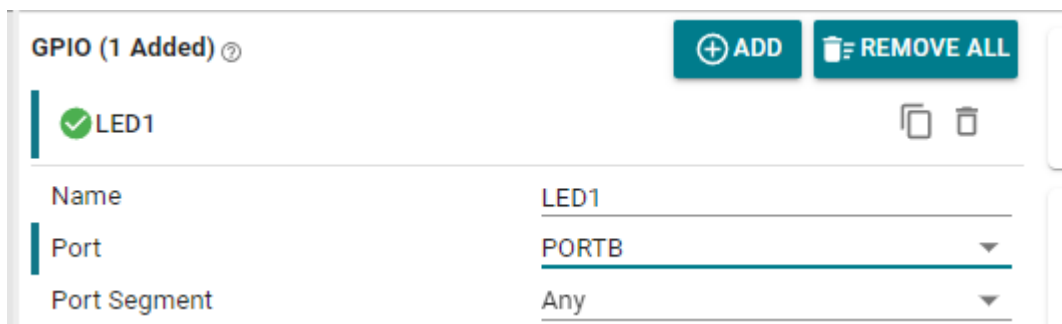


2. Parameter Settings

2.1 Set Port Parameters

Configure GPIO parameters and name the pin LED1.

From the LED part schematic, LED1 is on port PB2, so set Port to PORTB.



Parameter description:

Name: Custom name for the GPIO instance. By default, the name starts with the numeric suffix "0". You can customize the name to reflect the module's purpose (for example, naming the GPIO "LED1" so you know this pin is specifically for controlling LED1).

Port: The GPIO instance's port. LED is connected to GPIOB2 pin, so only PORTB can be selected.

Port Segment: Set port pull-up/down resistor. Note this is port pull-up/down, which sets the entire GPIOB port.

2.2 Set Pin Parameters

Set the pin to output mode, set the pin to output low level by default, set the pin to pull-down mode, set the pin to GPIOB_2, do not enable interrupt events.

1 added

+

ADD

🗑️

REMOVE ALL

✓

PIN_2

🗑️

Name	PIN_2
Direction	Output
Initial Value	Cleared
IO Structure	Any

Digital IOMUX Features

Internal Resistor

Pull-Down Resistor

Invert

Disabled

Drive Strength Control

Low

High-Impedance

Disabled

Assigned Port	PORTB
Assigned Port Segment	Any
Assigned Pin	2

Interrupts/Events

Event Subscribing Channel

Disabled (0)

Output Policy

Bit will be Set

Parameter description:

Name: User-defined pin name, set to PIN_2.

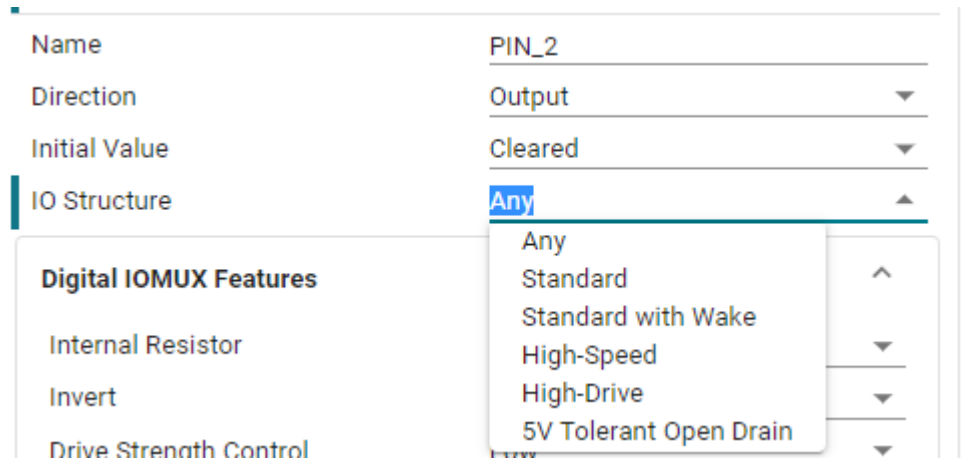
Direction: Set pin mode, input and output. Here we are controlling the LED light, so select output mode.

Name	PIN_2
Direction	Output
Initial Value	
IO Structure	Any

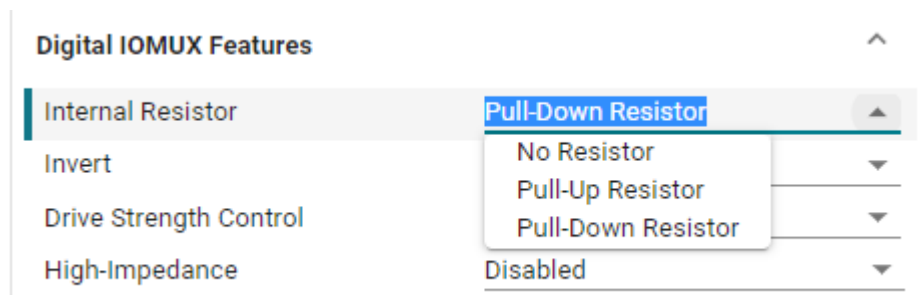
Initial Value: Set the initial state of the pin, can only be set when configured as output mode. There are two options, clear and set. Clear means output low level, set means output high level.

Name	PIN_2
Direction	Output
Initial Value ②	Cleared
IO Structure	

IO Structure: Set IO structure. There are multiple options: Default (Any), Standard, Wake (Standard with Wake), High-Speed, and 5V Tolerant Open Drain.



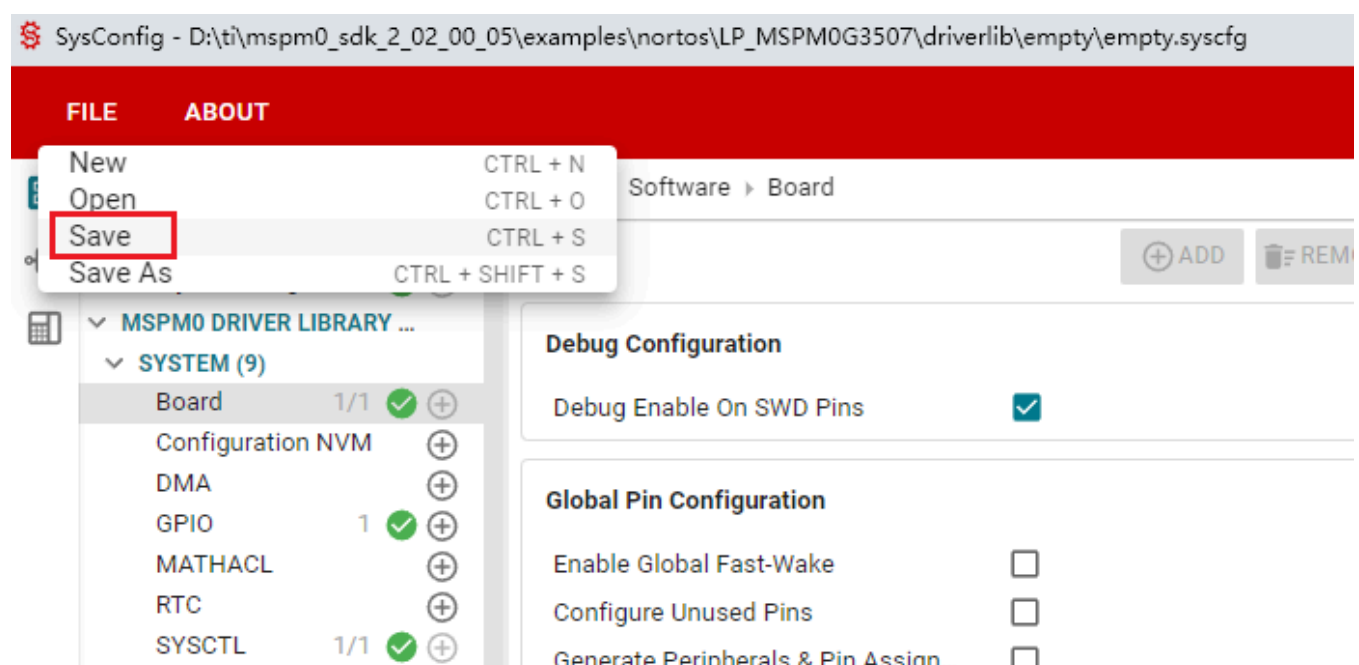
Internal Resistor: Set pin pull-up/down resistor. There are three options: none, pull-up, and pull-down resistor. Based on the LED connection method, pull-down resistor is selected here.



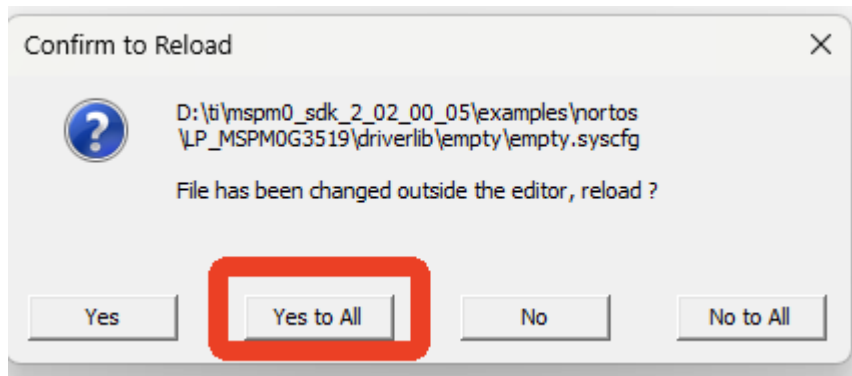
Assigned Pin: Set pin number. Enter the corresponding pin number for whichever pin you want to control. For example, the LED on the development board is connected to GPIOB2, so enter 2.

2.3 Save and Update Configuration File

Click SAVE to save the SYSCONFIG configuration, then close SYSCONFIG and return to KEIL.



Select **Yes to All** in the popup.



Now you can see that the empty.syscfg file has been updated with our set parameters.

```

1  /**
2  * These arguments were used when this file was generated. They will be automatically applied on subsequent loads
3  * via the GUI or CLI. Run CLI with '--help' for additional information on how to override these arguments.
4  * @cliArgs --device "MSPM0G351X" --part "Default" --package "LQFP-48(P)" --product "mspm0_sdk@2.02.00.05"
5  * @v2cliArgs --device "MSPM0G3519" --package "LQFP-48(P)" --product "mspm0_sdk@2.02.00.05"
6  * @versions {"tool": "1.21.1+3772"}
7  */
8
9  /**
10 * Import the modules used in this configuration.
11 */
12 const GPIO = scripting.addModule("/ti/driverlib/GPIO", {}, false);
13 const GPIO1 = GPIO.addInstance();
14 const SYSCTL = scripting.addModule("/ti/driverlib/SYSCTL");
15
16 /**
17 * Write custom configuration values to the imported modules.
18 */
19 GPIO1.$name = "LED1";
20 GPIO1.port = "PORTB";
21 GPIO1.associatedPins[0].$name = "PIN_2";
22 GPIO1.associatedPins[0].internalResistor = "PULL_DOWN";
23 GPIO1.associatedPins[0].assignedPin = "2";
24
25 const Board = scripting.addModule("/ti/driverlib/Board", {}, false);
26 Board.peripheral.$assign = "DEBUGSS";
27 Board.peripheral.swclkPin.$assign = "PA20";
28 Board.peripheral.swdioPin.$assign = "PA19";
29
30 SYSCTL.peripheral.$assign = "SYSCTL";
31
32 const ProjectConfig = scripting.addModule("/ti/project_config/ProjectConfig", {}, false);
33 ProjectConfig.migrationCondition = true;
34
35 /**
36 * Pinnux solution for unlocked pins/peripherals. This ensures that minor changes to the automatic solver in a future
37 * version of the tool will not impact the pinnux you originally saw. These lines can be completely deleted in order to
38 * re-solve from scratch.
39 */
40 GPIO1.associatedPins[0].pin.$suggestSolution = "PB2";
41

```

We also need to confirm whether the .c and .h files are updated. Simply compile, and the compilation will automatically update to KEIL. If not updated, we can also copy the file content from SYSCONFIG.

FILE ABOUT RESTART

Type Filter Text... X

Software > Board

PROJECT CONFIGURATION...
Project Config... 1/1 ✓

MSPM0 DRIVER LIBRARY ...
SYSTEM (10)
Board 1/1 ✓
Configuration NVM
DMA
GPIO 1 ✓
IWD
MATHACL
RTC_B
SYSCTL 1/1 ✓
SYSTICK
WWD
ANALOG (4)
ADC12
COMP
DAC12
VREF
COMMUNICATIONS (6)
I2C
I2C - SMBUS
MCAN
SPI
UART
UART - LIN
TIMERS (6)
TIMER
TIMER - CAPTURE
TIMER - COMPARE
TIMER - PWM
TIMER - QE
Timer Fault
SECURITY (3)

Board

Debug Configuration
Debug Enable On SWD Pins ☒

Global Pin Configuration
Enable Global Fast-Wake ☐
Configure Unused Pins ☐
Generate Peripherals & Pin Assign... ☐
Generate Resource Usage CSV ☐

Voltage Configuration
Configure ADC VDDA ☐

Initialization Priority Configuration
Initialization Priority 0 SYSTICK - active
Initialization Priority 1 PWM
Initialization Priority 2 QE
Initialization Priority 3 CAPTURE
Initialization Priority 4 COMPARE
Initialization Priority 5 TIMER
Initialization Priority 6 IWD
Initialization Priority 7 I2C
Initialization Priority 8 i2cSMBUS
Initialization Priority 9 UART
Initialization Priority 10 uartLIN
Initialization Priority 11 SPI

Problems
There are no problems in the current design.

Generated Files
Filter: all

File name	Category	Include in build
ti_msp_dl_config.c	MSPM0 Driver Library	☒
ti_msp_dl_config.h	MSPM0 Driver Library	☒
Event.dot	MSPM0 Driver Library	☒
empty.syscfg	Configuration Script	☐

4 Total Files

MSPM0G3519 (Device)
LQFP-100(PZ) (Package)

FILE ABOUT RESTART

Type Filter Text... X

Software > Board

PROJECT CONFIGURATION...
Project Config... 1/1 ✓

MSPM0 DRIVER LIBRARY ...
SYSTEM (10)
Board 1/1 ✓
Configuration NVM
DMA
GPIO 1 ✓
IWD
MATHACL
RTC_B
SYSCTL 1/1 ✓
SYSTICK
WWD
ANALOG (4)
ADC12
COMP
DAC12
VREF
COMMUNICATIONS (6)
I2C
I2C - SMBUS
MCAN
SPI
UART
UART - LIN
TIMERS (6)
TIMER
TIMER - CAPTURE
TIMER - COMPARE
TIMER - PWM
TIMER - QE
Timer Fault
SECURITY (3)

Board

Debug Configuration
Debug Enable On SWD Pins ☒

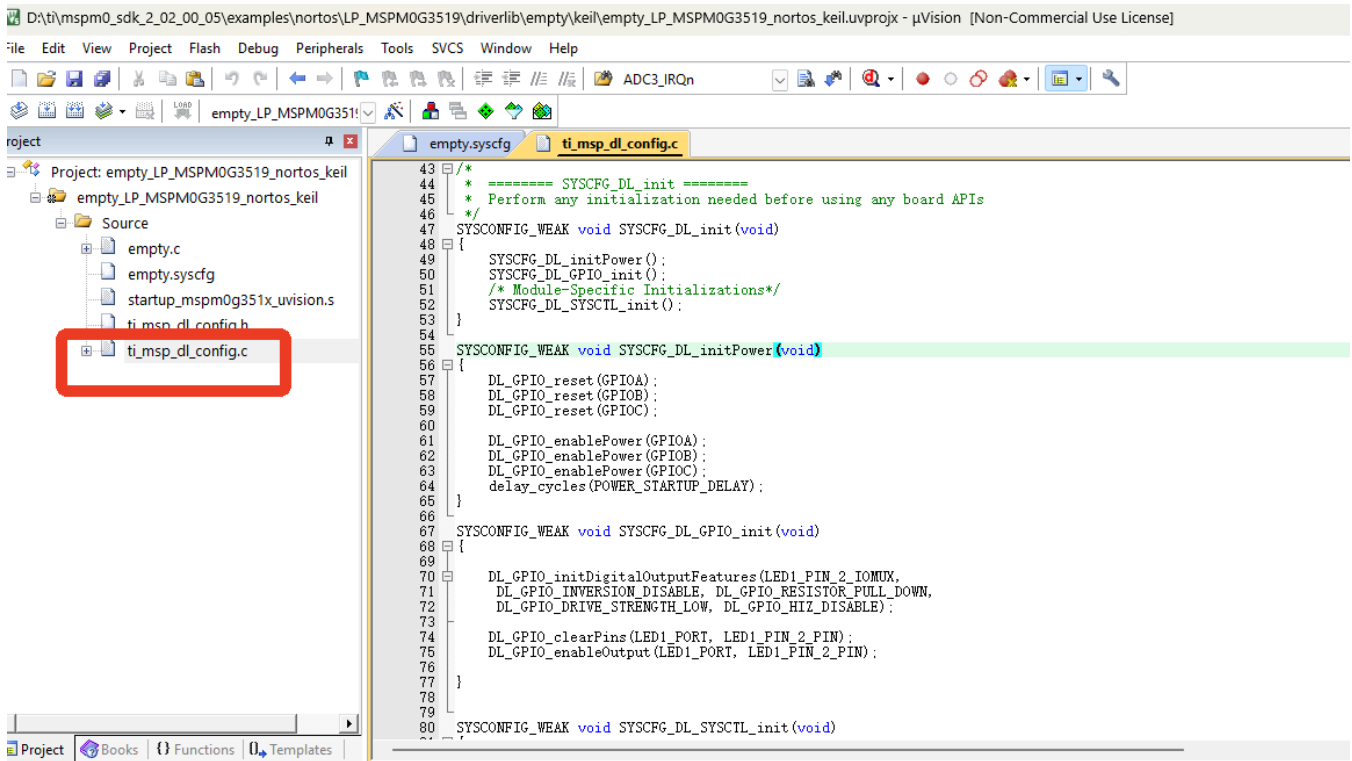
Global Pin Configuration
Enable Global Fast-Wake ☐
Configure Unused Pins ☐
Generate Peripherals & Pin Assign... ☐
Generate Resource Usage CSV ☐

Voltage Configuration
Configure ADC VDDA ☐

Initialization Priority Configuration
Initialization Priority 0 SYSTICK - active
Initialization Priority 1 PWM
Initialization Priority 2 QE
Initialization Priority 3 CAPTURE
Initialization Priority 4 COMPARE
Initialization Priority 5 TIMER
Initialization Priority 6 IWD
Initialization Priority 7 I2C
Initialization Priority 8 i2cSMBUS
Initialization Priority 9 UART
Initialization Priority 10 uartLIN
Initialization Priority 11 SPI

ti_msp_dl_config.c

```
1-/*
2- * Copyright (c) 2023, Texas Instruments Incorporated
3- * All rights reserved.
4- *
5- * Redistribution and use in source and binary forms,
6- * modification, are permitted provided that the following
7- * are met:
8- *
9- * Redistributions of source code must retain the at
10- * notice, this list of conditions and the following
11- *
12- * Redistributions in binary form must reproduce the
13- * notice, this list of conditions and the following
14- * documentation and/or other materials provided wit
15- *
16- * Neither the name of Texas Instruments Incorporate
17- * its contributors may be used to endorse or promot
18- * from this software without specific prior writer
19- *
20- * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS A
21- * AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BL
22- * THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNES
23- * PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRI
24- * CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INC
25- * EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
26- * PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF
27- * OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
28- * WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCL
29- * OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
30- * EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
31- */
32-
33-/*
34- * ===== ti_msp_dl_config.c =====
35- * Configured MSPM0 DriverLib module definitions
36- *
37- * DO NOT EDIT - This file is generated for the MSPM0
```



The .h file is the same. You can copy the content from SYSCONFIG.

3. Write Program

In the empty.c file, write the following code

```

#include "ti_msp_dl_config.h"

//Custom delay (not precise)
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    // The number of nested loops below is roughly calculated based on the master control
    frequency and the instruction cycle generated by the compiler, and needs to be adjusted
    through actual testing to achieve the required delay.
    for (i = 0; i < ms; i++)
    {
        for (j = 0; j < 8000; j++)
        {
            // Perform only one operation that is simple enough to predict its execution
            time
            __asm__("nop"); // "nop" stands for "no operation", which on most architectures
            consumes one or a few clock cycles
        }
    }
}

int main(void)
{
    SYS_CFG_DL_init();
    while (1)
    {

```

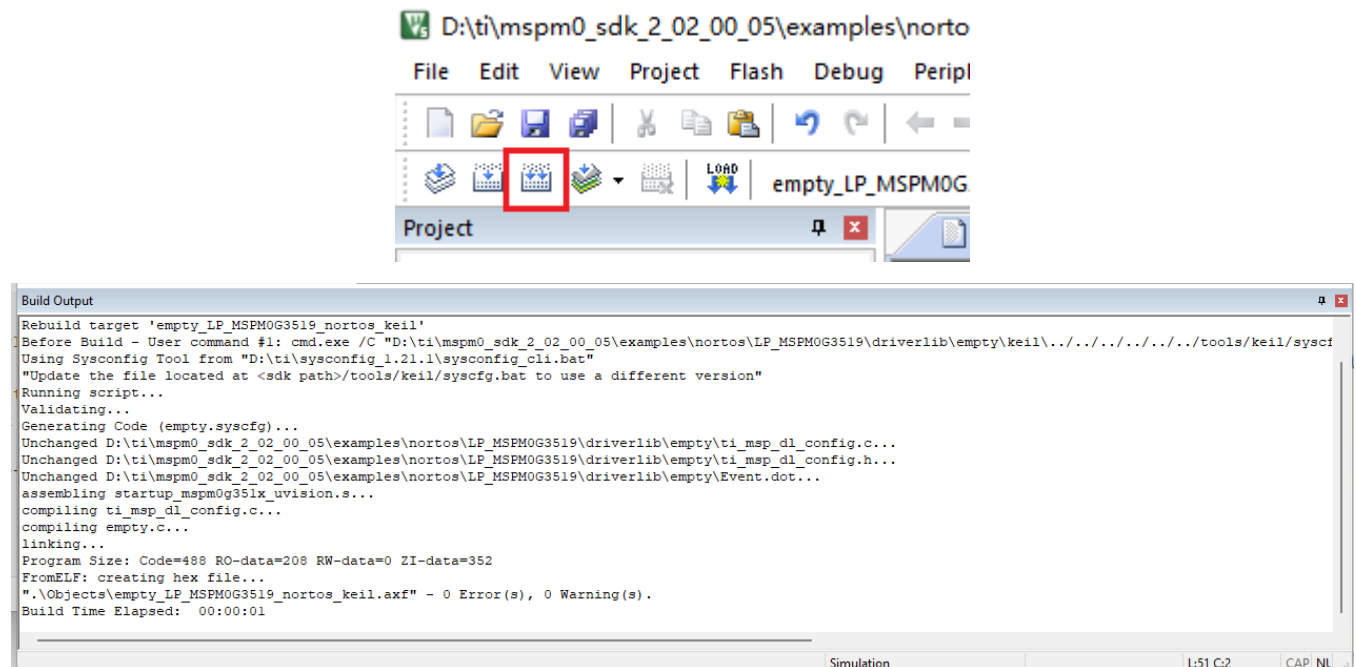
```

DL_GPIO_clearPins(LED1_PORT,LED1_PIN_2_PIN);//Output low level
delay_ms(1000);//Delay about 1S
DL_GPIO_setPins(LED1_PORT,LED1_PIN_2_PIN); //Output high level
delay_ms(1000);//Delay about 1S
}
}

```

4. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



4. Program Analysis

- dl_gpio.h

```

/**
 * @brief      Set a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to set high. Bitwise OR of @ref DL_GPIO_PIN.
 */
__STATIC_INLINE void DL_GPIO_setPins(GPIO_Regs* gpio, uint32_t pins)
{
    gpio->DOUTSET31_0 = pins;
}

/**
 * @brief      Clear a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to clear. Bitwise OR of @ref DL_GPIO_PIN.
 */
__STATIC_INLINE void DL_GPIO_clearPins(GPIO_Regs* gpio, uint32_t pins)
{

```

```

    gpio->DOUTCLR31_0 = pins;
}

/**
 * @brief      Toggle a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to toggle. Bitwise OR of @ref DL_GPIO_PIN.
 */
__STATIC_INLINE void DL_GPIO_togglePins(GPIO_Regs* gpio, uint32_t pins)
{
    gpio->DOUTTGL31_0 = pins;
}

```

__STATIC_INLINE void DL_GPIO_setPins(GPIO_Regs* gpio, uint32_t pins): This function controls pins to output high level.

__STATIC_INLINE void DL_GPIO_clearPins(GPIO_Regs* gpio, uint32_t pins): This function controls pins to output low level.

__STATIC_INLINE void DL_GPIO_togglePins(GPIO_Regs* gpio, uint32_t pins): This function toggles pin level. If it was high level, it becomes low level. If it was low level, it becomes high level.

- empty.c

```

int main(void)
{
    SYSCFG_DL_init();
    while (1)
    {
        DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //Output low level
        delay_ms(1000); //Delay about 1s
        DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //Output high level
        delay_ms(1000); //Delay about 1s
    }
}

```

Set PB2 to output low level, delay about 1s, then output high level, then delay about 1s. Repeat this process.

5. Experimental Phenomena

After the program is downloaded, you can see LED1 on the MSPM0 development board light up and blink approximately every 1 second.

